Targeted C++ practice - study priorities

Revised to separate explicit TA-note priorities from supplementary revision. These are original practice exercises, not confirmed exam questions.

Part A: TA-emphasized topics - do these first

A1 Function pointers and callbacks (page 2). A2 Operator overloading, const and return types (page 3). A3 Abstract interfaces and virtual functions (page 4). A4 Writing and tracing function templates (page 5).

Evidence: your C++ Concept Review.md notes explicitly emphasize writing function pointers, understanding virtual/abstract interfaces, writing operators and explaining const, and writing/tracing function templates. The specific exercises and callback counting task were designed for practice.

Part B: supplementary - exam inclusion not confirmed

B1 Const pointers; B2 Constructor initialization/static state; B3 Lifetime; B4 Array bounds; B5 Strings; B6 Ownership; B7 Stream formatting; B8 Stream recovery; B9 Optional std::vector/iteration (pages 6-14).

These address your questions and earlier mistakes, or broader lecture material. Some support Part A and some appear in tutorial material, but that does not confirm their exam inclusion. This label does not mean the TA excluded them.

If time is short, finish A1-A4 first. Use B1-B2 when const or initialization blocks your understanding of A2. Treat B9 as optional: Lecture 9 uses capital-V Vector examples; std::vector<int> was my supplementary exercise choice.

How to use this booklet

Assume C++17, required standard headers, and using namespace std unless qualified. No numerical memory addresses are requested. Write complete code and explain requested reasoning. Questions and solutions use matching A/B identifiers. This is targeted practice, not a timed or scored mock exam.

Sources for the distinction: C++ Concept Review.md (your TA notes); tutorial review photographs as summarized in 2026-10-26_Tutorial_Review.md; Lectures/9-CSC3002_Template and Iterator.pdf (broader iteration coverage).

A1. Function pointers: pass a rule into a loop

Write complete definitions. Use the supplied predicate inside countIf, rather than hard-coding a particular test.

(a) Write bool isPositive(long x). Declare an explicitly typed function-pointer variable test, initialized to isPositive. Explain the difference between isPositive and isPositive(5).

```
int countIf(const long* a, int n,
            bool (*pred)(long));
```

(b) Implement countIf. Assume n >= 0, pred is valid, and a refers to n valid elements when n > 0. It must return zero for n = 0.

(c) For {-3, 0, 4, 7}, call countIf using test and state the result. Then write isEven(long) and call the SAME countIf to count even values. State that result, too.

A2. Money operators: types, const and *this

Use member operators. Assume integer sums do not overflow.

```
struct Money {
    int cents;
    // Implement both operators here.
};
```

(a) Implement operator+ and operator+=. Both take a const-reference right operand. + must work on a const left operand and return a new Money without modifying inputs. += must modify the left operand and support chaining.

(b) Explain why return cents + rhs; and return cents + rhs.cents; do not work in this operator+. Explain what this and *this denote, and why += returns *this.

(c) Start with Money a{80}, b{15}, c{5}; then Money d = a + b; and (a += b) += c;. Give final cents for a, b, c, d. Does Money const operator+(...) replace trailing const?

A3. Abstract interfaces and exact overrides

Use this interface. tag is deliberately nonvirtual.

```
struct Reporter {
    virtual int report() const = 0;
    int tag() const { return 1; }
    virtual ~Reporter() = default;
};
```

(a) Write QuietReporter publicly derived from Reporter. report returns 0; tag returns 9. Use override where appropriate. Why can Reporter itself not be instantiated?

(b) Given QuietReporter s; Reporter& r = s;, write the results of r.report(), r.tag(), and s.tag(). Explain which calls depend on dynamic object type.

(c) Explain why const int report() does not override the required method. Explain the purpose of the virtual destructor when deleting a QuietReporter through Reporter*. Can derived code directly access private base members?

A4. Function templates: write, deduce and trace

Write code and explain types. Do not assume a double destination changes earlier arithmetic.

(a) Write template<typename T> T third(T x) returning x / 3. For auto a = third(8); auto b = third(9.0); double c = third(8); auto d = third<double>(9); give each type and value.

```
template<typename T>
T larger(T a, T b) { return a > b ? a : b; }
```

(b) Explain why larger(2, 2.5) fails deduction. Write a call with an explicit template argument that accepts both values and give its result.

(c) Write template<typename T> T addOne(T x). Return x + 1. Give the type and value of addOne(4) and addOne(1.5). Explain why assigning third(8) to a double cannot recover the discarded fraction.

B1. Const pointers: what is protected?

Consider each statement independently, starting from the declarations below.

```
int a = 12, b = 30;
const int* p = &a;
int* const q = &a;
```

(a) Mark each as valid or invalid and explain what is const: p = &b; *p = 7; q = &b; *q = 7; a = 7;

(b) Write a declaration of r that prevents both reseating r and changing a through r. Initialize r to point to a.

(c) Write int readValue(...) taking a pointer through which the int cannot be modified. Return the pointed-to value; assume the pointer is valid. Explain whether the caller may still modify a directly.

B2. Constructor initialization and static state

Complete the class below. Do not change the member declarations.

```
struct Ticket {
    const int code;
    static int created;
    // Constructor and getCode go here.
};
```

(a) Write Ticket(int c) using a member initializer list. Increment created in the body. Write getCode(), callable on const Ticket, and define created outside the class with initial value zero.

(b) Explain why code(c) is not a function call here. Why does code = c inside the constructor body fail?

(c) For Ticket a(9); const Ticket b(14); write a.getCode(), b.getCode(), and Ticket::created. Is created stored separately in each object?

B3. Object lifetime and dangling pointers

Classify behavior without inventing numerical memory addresses.

```
int* makeBad() {
    int value = 11;
    return &value;
}
int* q = makeBad();
cout << *q;
```

(a) Explain when value begins and ends its lifetime, what q stores, and why reading *q has no guaranteed output. Is this necessarily a compilation error?

(b) Rewrite makeBad and the caller so the caller safely receives the number by value. Do not allocate dynamically.

(c) Write int* identity(int& x) returning a pointer to x. Use it with an int in main and print through the pointer while that int is alive. Explain why this differs from makeBad.

B4. Array parameters and one-past boundaries

Assume every pointer passed to sum refers to at least n elements when n > 0.

```
int sum(const int a[], int n);
```

(a) Implement sum using the explicit length. It must handle n = 0. Explain why sizeof(a) / sizeof(a[0]) inside this function cannot recover the caller's array length.

```
int values[] = {2, 4, 6, 8};
int* end = values + 4;
```

(b) Classify independently: forming values + 4; reading *end; reading *(end - 1). Give a value only where the read is valid.

(c) Rewrite the sum loop using a pointer p and the condition p != end. Use the four-element values array above. Never dereference the one-past pointer.

B5. C-strings: capacity, terminators and concatenation

Assume the standard string and C-string headers are available.

```
char label[] = "Go";
string word = "Go";
```

(a) State sizeof(label), strlen(label), and the minimum char-array capacity needed for "Blue". Explain why strcpy(label, "Blue") is unsafe.

(b) Write a safe fixed-size char array containing "Blue". Then write a std::string version initially containing "Go" and replace its contents with "Blue".

(c) Classify each independently and give the result when valid: "Hi" + "!"; string("Hi") + "!"; "Hi" + string("!"); strcmp("Go", "Go"). Explain why strcpy does not resize label.

B6. Ownership: raw aliases and smart pointers

Consider each subpart independently. All allocations succeed.

```
int* p = new int(6);
int* q = p;
delete p;
p = nullptr;
```

(a) Describe p and q after these statements. May you read *q or delete q? Write a version that clears both local aliases after one deletion.

```
auto s = make_shared<int>(6);
weak_ptr<int> w = s;
```

(b) Give s.use_count(). Write code using w.lock() to print the value only if it is still alive. Then explain w.expired() after s.reset(), assuming no other shared_ptr survives.

(c) Write code that creates a unique_ptr<int>, transfers ownership to another unique_ptr, and checks that the source is empty. Also state the matching deletion for new int[1], and explain why copying an owning raw pointer is not a deep copy.

B7. Streams: exact output and persistent formatting

Write exact characters, including underscores and decimal zeros.

```
istringstream in("18\nLin Chen\n");
int id;
string first, name;
in >> id;
getline(in, first);
getline(in, name);
ostringstream out;
out << fixed << setprecision(2);
out << setfill('_') << setw(7) << 4.5;
out << '|' << 8.0;
cout << id << ":[" << first << "]\n";
cout << '[' << name << "]\n";
cout << out.str() << '\n';
```

(a) Write all three output lines. Explain the first getline result and count the padding characters.

(b) State which persist after printing 4.5: width, fill, fixed, precision. Does out.str() alone print or empty the stream?

(c) Rewrite the input section to read id and then the full name into name using ONE getline. Discard the rest of the id line with ignore and numeric_limits.

B8. Recover from failed numeric input

The input contains exactly one nonnumeric character between valid integers.

```
istringstream numbers("5 15 x 25");
int n, total = 0;
while (numbers >> n) total += n;
```

(a) State total after the loop and identify the unread character blocking the next integer extraction. Why does the loop body not process x?

(b) Write the recovery: reset the error flags, consume the one bad character, and add the remaining integer only if extraction succeeds. Give the new total.

(c) Explain why clear() alone is insufficient. Explain why sizeof(n) != 4 cannot identify bad input, and why casting n to int cannot repair a failed stream.

B9. Optional standard-library vector and iteration

Supplementary only. std::vector<int> is an exercise choice, not a confirmed TA exam clue. Lecture 9 uses capital-V Vector examples; do not assume the two APIs are identical.

```
std::vector<int> v{2, 5, 8};
```

(a) Write a range-for loop adding 3 to each actual element. Give the resulting vector. Explain the difference between auto x and auto& x.

(b) Starting again from {2, 5, 8}, write an iterator loop printing each element. Explain why end() must not be dereferenced.

(c) Rewrite the traversal using int values[] = {2, 5, 8}; and a pointer. Use values + 3 as the stopping boundary and never read through it.